

Тестовое задание

Trainee Frontend Developer / React · Next.js

Уровень: Trainee Frontend-разработчик

Стек: React 18+, JavaScript (или TypeScript), CSS/SCSS, Zustand или Redux Toolkit

Формат сдачи: GitHub-репозиторий + ссылка на превью

Это задание — не экзамен. Мы не ждём идеального кода и знания всех паттернов. Мы хотим посмотреть, как ты думаешь: можешь ли ты разобраться с незнакомой задачей, написать рабочий компонент и объяснить своё решение.

Тебе предстоит собрать экран «Стоп-лист кухни» — тот самый список позиций, которые сегодня нельзя продавать, потому что закончились продукты. Это реальная фича из нашего продукта, упрощённая до учебного масштаба. Сделай её рабочей — аккуратный код важнее большого объёма.

Не старайся охватить всё. Лучше сдать меньше — но аккуратно.

1. Задача

Одностраничное приложение из двух блоков: слева меню ресторана (список позиций), справа — стоп-лист. Позицию можно отправить в стоп-лист через форму с причиной и вернуть обратно в меню. Данные меню берутся из локального JSON-файла, бэкенда нет.

Что нужно реализовать

Часть фичи	Требования
Список меню	Минимум 10 позиций из локального <code>menu.json</code> . Карточка показывает название, категорию, цену и остаток на кухне (число порций). Позиции, уже находящиеся в стоп-листе, в списке меню не показываются.
Поиск и фильтр	Поле поиска по названию (без учёта регистра) и фильтр по категории — «Все», «Кухня», «Бар», «Десерты». Работают вместе: поиск применяется внутри выбранной категории. Если ничего не найдено — текст «Ничего не найдено», а не пустой экран.
Форма «В стоп-лист»	По кнопке на карточке открывается форма: причина (выпадающий список: «Закончились продукты», «Плохое качество партии», «Нет повара на станции», «Другое»), комментарий, время предполагаемого возврата. Форма валидируется по правилам из раздела 3.
Стоп-лист	После отправки формы позиция появляется в правом блоке: название, причина, комментарий и время возврата. У каждой позиции — кнопка «Вернуть в меню», которая убирает её из стоп-листа и возвращает в список слева.
Счётчик и состояние	Над стоп-листом — счётчик: «В стоп-листе: 3 из 12». Если стоп-лист пуст — сообщение «Все позиции в продаже».
Адаптивность	Корректный вид на мобильном (375 px) и десктопе (1280 px). На мобильном блоки идут друг под другом. Достаточно базовых медиа-запросов — pixel-perfect не нужен.

2. Технические требования

Обязательный стек

- **React 18+** — только функциональные компоненты и хуки (useState, useEffect, при необходимости useMemo)
- **Vite или Next.js** как сборка/каркас проекта — на твой выбор, оба варианта принимаются
- **CSS/SCSS** — своя вёрстка, без UI-китов вроде Bootstrap или MUI. SCSS приветствуется: вложенность, переменные для цветов и отступов
- **Zustand или Redux Toolkit** — стоп-лист и список меню хранятся в общем сторе, а не прокидываются пропсами через все компоненты
- **Разделение на компоненты** — минимум: карточка позиции, список, форма, элемент стоп-листа

Если ты ещё не уверен в Zustand — не страшно. Он небольшой: стор, пара полей состояния и функции, которые их меняют. Используй его там, где понятно, как это сделать. Нам важен сам факт того, что состояние живёт в одном месте, а не размазано по компонентам.

Приветствуется, но не обязательно

- TypeScript хотя бы для описания позиции меню и пропсов компонентов
- Сохранение стоп-листа в localStorage, чтобы он переживал перезагрузку страницы
- Плавные переходы: появление позиции в стоп-листе, открытие формы (CSS-transition или Framer Motion)
- ESLint и Prettier с конфигом в репозитории

Структура проекта (ориентир)

Необязательно следовать точно — главное, чтобы структура была понятна:

```
src/  
  components/  
    MenuList.jsx      # список позиций меню  
    MenuItemCard.jsx  # карточка одной позиции  
    StopListForm.jsx  # форма отправки в стоп-лист  
    StopList.jsx      # правый блок со стоп-листом  
    StopListItem.jsx  # одна позиция в стоп-листе  
    Filters.jsx       # поиск и фильтр по категории  
  store/  
    useStopListStore.js # Zustand-стор: меню, стоп-лист, действия  
  data/  
    menu.json          # мок-данные меню (10+ позиций)  
  utils/  
    validation.js      # правила валидации формы  
  styles/  
    variables.scss     # цвета, отступы, брейкпоинты  
    app.scss  
  App.jsx  
  main.jsx
```

Пример типов / контрактов

Можно использовать этот шаблон или написать свой:

```
// Позиция меню — из menu.json
type MenuItem = {
  id: number;
  name: string;           // «Том ям с креветками»
  category: 'kitchen' | 'bar' | 'dessert';
  price: number;          // в рублях
  portionsLeft: number;   // остаток порций на кухне
};

// Запись в стоп-листе
type StopListEntry = {
  itemId: number;
  reason: 'out_of_stock' | 'bad_quality' | 'no_cook' | 'other';
  comment: string;        // может быть пустым, кроме причины «Другое»
  returnAt: string;       // '18:30'
  createdAt: string;      // ISO-строка, момент добавления
};
```

Вынеси типы в отдельный файл — это хорошая практика, которую мы ценим. Но даже если не вынес — важно, что структура данных продумана.

3. Валидация

Форма проверяется при отправке; ошибки показываются под соответствующими полями и не дают отправить форму, пока не исправлены. Если пользователь начал исправлять поле — ошибку по нему можно убирать сразу.

Поле	Правило
Причина	Обязательное. По умолчанию ничего не выбрано — выбор «— выберите причину —» отправку не проходит.
Комментарий	Необязательный, максимум 200 символов. Становится обязательным (минимум 10 символов), если выбрана причина «Другое».
Время возврата	Обязательное. Формат ЧЧ:ММ, значение от 00:00 до 23:59. Можно использовать <code>input type="time"</code> — тогда достаточно проверить, что поле не пустое.
Повтор	Одна и та же позиция не может попасть в стоп-лист дважды. Если позиция уже там — кнопка на карточке недоступна.

Пример: как проверить время возврата

Можно использовать эту функцию или написать свою:

```
export function isValidTime(value) {
  if (!value) return false;
  const match = value.match(/^(\\d{2}):\\d{2})$/);
  if (!match) return false;
  const hours = Number(match[1]);
  const minutes = Number(match[2]);
  return hours >= 0 && hours <= 23 && minutes >= 0 && minutes <= 59;
}
```

Не обязательно обрабатывать все edge cases идеально — достаточно, чтобы базовые сценарии работали корректно.

4. UX и дизайн

Мы не ждём дизайна уровня Dribbble. Нам важна аккуратность: поля выровнены, ошибки видны, кнопка реагирует на отправку.

Цветовая схема	Любая нейтральная палитра. Можно взять за основу: фон #F6F3EE, акцент #C6462F, текст #171512.
Типографика	Системный шрифт (sans-serif). Иерархия: заголовок → лейблы полей → текст ошибок.
Ошибки	Красный текст под полем. Поле с ошибкой — красная обводка. Не <code>alert()</code> .
Лoader	Текст состояния или крутящийся индикатор. Кнопка disabled во время отправки.
Стоп-лист	Позиция в стоп-листе визуально отличается от карточки меню: приглушённый фон, зачёркнутое название или бейдж с причиной — на твой вкус.

Простое и рабочее лучше, чем красивое и сломанное.

5. Сдача задания

Формат

- Создай публичный репозиторий на GitHub.
- Задеплой проект на Vercel (бесплатный план).
- Пришли ссылку на репозиторий и ссылку на задеплойенный проект.

i Проще всего: залить репозиторий на GitHub, зайти на vercel.com, нажать Add New → Project и выбрать репозиторий. Vite и Next.js Vercel определяет сам, ничего настраивать не нужно.

README.md — обязательно

- Как запустить проект локально (команды: `npm install`, `npm run dev`)
- Какие решения ты принял и почему (2–3 предложения)
- Что бы ты доделал, если бы было больше времени

README — часть задания. Умение коротко объяснить своё решение так же важно, как сам код.

Чего делать не нужно

- Реальный бэкенд, база данных или запросы к серверу — данные берутся из локального JSON
- Авторизация, роли и личный кабинет
- Pixel-perfect вёрстка и сложный дизайн — достаточно аккуратного и читаемого интерфейса
- Тесты — будут плюсом, но не обязательны

6. Критерии оценки

Мы оцениваем не только результат, но и подход. Рабочая и понятная реализация важнее попытки сделать «идеально» с незаконченным результатом.

Критерий	Вес	На что смотрим
Работоспособность	40%	Позиция уходит в стоп-лист и возвращается обратно, поиск и фильтр работают вместе, валидация не пропускает пустую форму — всё основное работает без критических ошибок
Качество кода	25%	Понятная структура, разбиение на компоненты, читаемые имена переменных, нет очевидного дублирования
Zustand / Redux	15%	Состояние живёт в сторе, а не дублируется в компонентах; действия названы понятно; нет «прокидывания» пропсов через три уровня
UX и адаптивность	10%	Интерфейс не разваливается на 375 px, ошибки видны, пустые состояния обработаны
README	10%	Понятно, как запустить; написано, какие решения приняты и что осталось

Бонусные баллы

- Стоп-лист сохраняется в `localStorage` и переживает перезагрузку страницы
- Юнит-тест на функцию валидации времени или на логику добавления позиции в стоп-лист
- Аккуратные анимации появления и удаления позиций
- TypeScript с описанными типами позиции меню и записи стоп-листа

COPERTO — AI-первая студия для HoReCa • Вопросы: в чат с рекрутером